CAPSTONE PROJECT REPORT

MINI-COURIER PLANNER

Submitted by: _____

Roll number: _____

Course / Section: _____

Institution: _____

Academic year: 2026

Project based on the Capstone Assignment: Mini-Courier Planner

1. Problem Statement

The project develops a prototype courier-planning system. Given parcels, a carrying-capacity limit, and a weighted delivery network, the application prioritizes parcels, chooses a valuable feasible parcel set, calculates shortest paths, and estimates an overall delivery tour.

2. Objectives

The objectives are to prioritize parcels by deadline and priority, select parcels under a vehicle capacity, find shortest paths in a weighted graph, generate an approximate tour, and measure algorithm execution time.

3. System Design

The application is modular. Sorting, knapsack, Dijkstra, TSP, data loading, visualization, benchmarking and integration are implemented in separate Python modules. CSV files store parcel and road data.

4. Methodology — Sorting

Parcels are sorted by earliest deadline first. When deadlines match, HIGH priority precedes MEDIUM and LOW. Parcel ID is the final deterministic tie-breaker. Python Timsort has O(n log n) worst-case time complexity.

5. Methodology — Dynamic Programming

The 0/1 Knapsack algorithm maximizes total parcel value without exceeding the van capacity. Each parcel can be selected at most once. Its time and space complexity are O(nW), where n is parcel count and W is integer capacity.

6. Methodology — Dijkstra

The road network is represented as an adjacency dictionary. Dijkstra's algorithm uses a min-priority queue to find shortest paths with non-negative edge weights. With a binary heap, its complexity is O((V+E) log V).

7. Methodology — TSP Approximation

The nearest-neighbour heuristic starts at DEPOT, repeatedly visits the closest unvisited delivery point using shortest-path distances, and returns to the depot. It is a greedy approximation and does not guarantee the globally shortest tour.

8. Sample Data and Testing

The supplied sample contains eight parcels and eight delivery points (A-H), with a default van capacity of 20 kg. The road dataset is a weighted undirected graph. The benchmark tests input sizes 10, 30, 60 and 100 using generated synthetic data.

9. Complexity Summary

Sorting: O(n log n). Knapsack: O(nW). Dijkstra: O((V+E) log V). Nearest-neighbour with repeated Dijkstra: approximately $O(k(V+E) \log V + k^2)$, where k is the number of delivery points.

10. Conclusion

The prototype integrates sorting, dynamic programming, graph algorithms and an approximation technique in one courier-planning application. It demonstrates a practical baseline while making the trade-offs of greedy route planning clear.

11. Future Scope

Future improvements include traffic-aware travel costs, one-way roads, multiple vehicles, delivery time windows, 2-opt route improvement, and live map data.

Sample execution output

```
============================================================
MINI-COURIER PLANNER
============================================================
1. PARCEL PRIORITIZATION (earliest deadline first)
P01: HIGH, deadline 10:30, 5 kg, destination A
P03: HIGH, deadline 11:00, 8 kg, destination C
P02: MEDIUM, deadline 12:00, 3 kg, destination B
P05: HIGH, deadline 13:00, 6 kg, destination E
P06: MEDIUM, deadline 14:00, 4 kg, destination F
P07: MEDIUM, deadline 15:00, 7 kg, destination G
P04: LOW, deadline 16:00, 2 kg, destination D
P08: LOW, deadline 17:00, 3 kg, destination H
2. OPTIMAL PACKING (0/1 Knapsack DP)
P01 -> A (5 kg)
P02 -> B (3 kg)
P05 -> E (6 kg)
P06 -> F (4 kg)
P04 -> D (2 kg)
Total weight: 20/20 kg
Total parcel value: 350
3. DIJKSTRA SHORTEST-PATH EXAMPLE
To A: 4.0 km | Path: DEPOT -> A
4. NEAREST-NEIGHBOUR DELIVERY TOUR
DEPOT -> A -> B -> D -> F -> E -> DEPOT
Total tour distance (including return to depot): 29.0 km
5. MODULE RUNTIMES
Sorting: 0.3000 ms
Knapsack DP: 0.0338 ms
Dijkstra: 0.0219 ms
Nearest-neighbour TSP: 0.0711 ms
Note: Runtime values depend on the computer and current system load.
```

Note: runtime values are machine-dependent. Rerun the application and benchmark on the submission computer and discuss those measurements.